Ders 20 Genişletilmiş Notu: Özyinelemeli Fonksiyonlar: Faktöriyel, Hızlı Üs Alma, Recursion Tree, Merge Sort ve Final Projesi

Ders 20 Genişletilmiş Notu: Özyinelemeli Fonksiyonlar: Faktöriyel, Hızlı Üs Alma, Recursion Tree, Merge Sort ve Final Projesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Doktor ve Teşhis Sorusunun Kalan Kısımları
- 2. Özyinelemeye Dönüş: Stack ve Base Case
- 3. Özyinelemeli Faktöriyel
- 4. Özyinelemeli Üs Alma
- 5. Daha İyi Bir Üs Alma Fikri
- 6. Üs Almanın Diğer Yazımları
- 7. Ders Notundan Üç Fonksiyon
- 8. Recursion Tree (Özyineleme Ağacı)
- 9. Hızlı Üs Alma: power_v2
- 10. Çağrı Sayısını Ölçme Denemesi
- 11. Üssün İkili Açılımıyla Üs Alma
- 12. Karmaşıklık Adları ve power_v2'nin Analizi
- 13. Geçici Değişken Olmadan: power_v3
- 14. Merge Sort ve Quick Sort
- 15. Laboratuvardaki Özyineleme Soruları
- 16. Sonraki Konular
- 17. Final Projesinde Beklenen Algoritma Analizi

1. Doktor ve Teşhis Sorusunun Kalan Kısımları

Hoca derse, önceki dersten kalan doktor sorusunun cevabını kısaca özetleyerek başladı. Senaryoda D doktorun H hastaya koyduğu teşhisler bir matriste saklanıyordu. Bunun yanında her hastanın gerçek teşhisini tutan bir dizi vardı. Sorunun ilk üç kısmı önceki derste yapılmıştı.

1.1 Teşhis Kodları Neden Kategoriktir?

Hastalıkların 1 ile 100 arasında kodları vardır, ama bu kodlar kategoriktir. Hoca bunun sonucunu şöyle anlattı: 100. hastalık ile 80. hastalık arasında fark alıp anlamlı bir değer çıkaramayız. Bu yüzden doğrudan sayısal bir en yakın çift (closest pair) hesabı yapılamaz. Yapılabilecek tek karşılaştırma, iki kodun eşit olup olmadığına bakmaktır.

1.2 En Başarılı Doktor

Matrisin her satırı bir doktordur. En başarılı doktoru bulmak için her satır gerçek teşhis dizisiyle karşılaştırılır. Dış döngü doktorları, iç döngü hastaları gezer. Teşhis doğruysa sayaç

artar. Bu sayaç tek bir doktora aittir, bu yüzden iç döngü bitince skor dizisine yazılır. Bütün doktorların skorları bulunduktan sonra bu dizinin en büyük elemanı en başarılı doktoru verir.

Aşağıdaki parçada `M`, hocanın tahtada D dediği matristir. Doktor sayısıyla karışmasın diye burada farklı adla yazıldı.

```
for (i = 0; i < D; i++) {
    c = 0;
    for (j = 0; j < H; j++)
        if (M[i][j] == dogru[j])
            c++;
    skor[i] = c;
}
```

Hoca hazır çözümde teşhislerin kullanıcıdan kontrol edilmeden alındığını, önceki derste ise bunların rastgele girildiğini hatırlattı. Gösterdiği hazır çözümde bu kısım yoktu. Bu yüzden derste yazdıkları çözüme gönderme yapıp devam etti.

1.3 Birbirine En Çok Benzeyen İki Doktor

Bu kez bir doktoru gerçek teşhislerle değil, başka bir doktorla karşılaştırırız. Birinci doktor ikinci, üçüncü, dördüncü doktorlarla kıyaslanır, sonra ikinci doktor üçüncü, dördüncü ve sonrakilerle kıyaslanır. Bu yüzden iki doktor döngüsü ve bunların içinde hastaları gezen üçüncü bir döngü gerekir.

Hoca döngü sınırlarını öğrencilere sordu. `i` sıfırdan başlar. `j` ise `i + 1`'den başlamak zorundadır, çünkü bir doktoru kendisiyle karşılaştırmanın anlamı yoktur. `k` hastaları 0'dan $H-1$'e kadar gezer. İki doktorun aynı hastaya koyduğu teşhis eşitse sayaç artar.

```
for (i = 0; i < D - 1; i++) {
    for (j = i + 1; j < D; j++) {
        c = 0;
        for (k = 0; k < H; k++)
            if (M[i][k] == M[j][k])
                c++;
        if (c > enc) {
            enc = c;
            i1 = i;
            i2 = j;
        }
    }
}
```

Hoca bütün benzerliklerin saklanmasına gerek olmadığını da belirtti. Geçici değişkenler kullanılır: o anki en iyi benzerlik ve onun iki doktor indisi tutulur. Sonraki bir kıyaslamada daha büyük bir değer bulununca bunlar güncellenir.

> **Ders sonrası not:** Hoca döngü sınırlarını konuşurken "$D-1$" ve "$D-2$" değerlerini söyledi. Sıfırdan başlayan indislerle `i` en fazla $D-2$, `j` en fazla $D-1$ olur. "$D-2$" büyük olasılıkla `i`'nin son değeri için söylendi. Yukarıdaki kod `gcc -Wall` ile uyarısız derlendi. `j` da $D-2$'de bitseydi son doktor hiç karşılaştırılmazdı.

1.4 Hazır Çözüm: Transpoz ve findDistance

Hoca ardından hazır bir çözümü inceledi. O çözümde en yakın iki doktor aranmadan önce matrisin transpozu alınmıştı. Hoca bunun olabileceğini, ama kendisine göre gerek olmadığını söyledi. Transpoz, satırlarla sütunların yer değiştirmesidir ve sonuç yeni bir matrise aktarılır.

Bunu C'de yapmanın bir sonucu vardır. C'de bir fonksiyon matrisi doğrudan döndüremez. Bu yüzden hem kaynak matris hem de doldurulacak yeni matris fonksiyona parametre olarak gönderilir. Fonksiyon yeni matrisin içini doldurur. Hoca bunu referansla çağırma (call by reference) olarak adlandırdı.

Transpozdan sonra çözüm, en yakın çift problemine döner:

- En küçük uzaklığı tutan değişken önce `INT_MAX` ile başlatılır.
- Aynı üç döngü kurulur: birinci eleman 0'dan n'e, ikinci eleman `i + 1`'den n'e, üçüncü döngü boyut sayısına kadar.
- Her karşılaştırmada kategorik `findDistance` fonksiyonu çağrılır. İki değer eşitse uzaklık yoktur, 0 döner. Farklıysa (biri 70., öteki 78. hastalığı verdiyse) 1 döner. Sayısal fark alınmaz.
- Toplam uzaklık önceki en küçükten küçükse değer yedeklenir ve `indis1`, `indis2` güncellenir.
- Uzaklık sıfırlanır ve bir sonraki ikiliye geçilir.

Hoca kodda görülen `static` kullanımına takılmamayı istedi. Bu kullanım diziyi doğrudan sıfırlamak içindi ve daha sonra anlatılacak. Onun yerine dizi tanımlanırken de sıfırlanabilir, ayrıca dizi tanımları fonksiyonun yukarısında yapılmalıdır.

1.5 Demokrasi Usulü Teşhis: Kimler Mağdur Olur?

Sorunun son kısmında teşhis, doktorların oylamasıyla konuyordu. Her hasta için en sık konan teşhis kabul edilir. Bu teşhis gerçek teşhisten farklıysa hasta mağdur olur. Hazır çözümde bu iş "kimler ölecek" adlı bir fonksiyondaydı. Adımlar şöyledir:

1. Matris fonksiyona gönderilir.
2. Bir hasta için bütün doktorların koyduğu teşhisler bir diziye alınır.
3. Bu dizi frekans bulma fonksiyonuna gönderilir. Her teşhisten kaç tane olduğu hesaplanır, yani histogram çıkarılır.
4. Frekans dizisi en büyüğü bulan fonksiyona gönderilir ve en sık konan teşhis bulunur.
5. En sık konan teşhis gerçek teşhisle karşılaştırılır. Aynı değilse o hasta mağdurdur ve mağdurlar dizisinin ilgili gözüne bayrak olarak 1 yazılır.
6. Döngüler bitince ana programa dönülür ve mağdurlar dizisi ekrana yazdırılır.

Hoca burada fazla bir algoritmik derinlik olmadığını söyledi. Yapılan şey, matrisleri ve dizileri fonksiyona göndermek ve fonksiyon içinde onlara yazmaktır. Mağdurlar dizisi bunun en basit örneğidir.

1.6 Genel Amaçlı Fonksiyonlar

`findDistance`, transpoz, frekans bulma, en büyüğü bulma gibi fonksiyonlar genel amaçlıdır. Frekans bulma yalnız doktor senaryosunda değil, başka senaryolarda da kullanılabilir. Hoca gelecek dönem bu tür fonksiyonların ayrı bir C dosyasına konulup her programda kullanılacağını söyledi. Nesneye dayalı programlamada ise bunlar tamsayı veya karakter için ayrı ayrı değil, daha genel amaçlı yazılacak. Böyle fonksiyonlar programın hacmini küçültür ve bir kez yazılıp tekrar tekrar çalıştırılır.

2. Özyinelemeye Dönüş: Stack ve Base Case

Hoca özyinelemeye (recursion) girmeden önce önceki dersi hatırlattı. Her fonksiyon çağrısında, programın doğru devam edebilmesi için bazı değerlerin saklanması gerekir. Öğrencilerin de söylediği gibi bu değerler stack bellekte saklanır.

Özyinelemeli fonksiyon da normal bir fonksiyon çağrısıdır. Tek farkı kendi kendini çağırmasıdır. Bir çıkış kodu yazılmazsa kendini sonsuza kadar çağırır. Bu çıkışa temel durum (base case) denir.

Base case genelde matematikten bilinir. Faktöriyelde her sayının faktöriyeli alınamaz: $0!$'in bir karşılığı vardır, negatif sayıların ise yoktur. $f(5)$ için şunu biliriz:

$$f(5) = 5 \cdot f(4) = 5 \cdot 4 \cdot f(3) = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot f(0)$$

Bunun sonsuza kadar gitmemesi için en fazla $f(0)$'a kadar yazılır. $f(0)$, normal işleyişten farklı çalışmalıdır: kendini tekrar çağırmaz, doğrudan değer döndürür.

Normal işleyişte gelen n değeri ile fonksiyonun $n-1$ için çağrısı çarpılır. Problem her adımda küçültülür. Hoca bunun azalt ve yönet tekniği olduğunu belirtti.

3. Özyinelemeli Faktöriyel

3.1 Recurrence Relation

Hoca bağıntıyı parçalı fonksiyon olarak yazdı. Bu ifadeye recurrence relation denir:

$$f(n) = \begin{cases} 1 & n = 0 \\ n \cdot f(n-1) & n > 0 \end{cases}$$

$n = 1$ de base case olarak yazılabilirdi, ama hoca faktöriyel tanımını zorlamamak için yalnız $n = 0$'ı aldı.

Bizim normal faktöriyel algoritmamız bir `for` döngüsüydü: 1'den n'e kadar sayılar `f = f * i` ile çarpılır ve n tane temel işlem yapılır. Hoca burada şu soruyu bıraktı: Özyineleme bu n'i azaltabilir mi? Dönem başından beri söylendiği gibi iyi algoritma, ya daha az adımda (daha kısa sürede) çalışan ya da daha az bellek kullanan algoritmadır. Bazı örneklerde özyinelemenin hiçbir farkının olmadığı, hatta dezavantajlı olduğu görülecek. Ama bazı problemlerde özyinelemeli yaklaşım işleri çok kolaylaştırır.

3.2 Kod

Önceki derste yazılan kodda base case 2 idi, iki ayrı şart vardı. Hoca bunu sadeleştirdi: n sıfırsa 1 döndür, değilse `n * fakt(n - 1)` döndür. Ayrıca fonksiyona ilk çağrıda negatif değer gönderilme olasılığı için en başa bir kontrol ekledi ve hata belirtmek için -1 döndürdü.

```c
#include <stdio.h>

int fakt(int n)
{
    if (n < 0)
        return -1;            /* hatalı giriş */
    if (n == 0)
        return 1;             /* base case */
    return n * fakt(n - 1);   /* recurrence relation */
}

int main(void)
{
    int i = fakt(3);

    if (i == -1)
        printf("Sonuc hatali\n");
    else
        printf("Sonuc: %d\n", i);
    i = fakt(-3);
    if (i == -1)
        printf("Sonuc hatali\n");
    else
        printf("Sonuc: %d\n", i);
    return 0;
}
```

Derste önce `fakt(3)` çalıştırıldı ve 6 bulundu. Sonra `main`'de dönen sonuç bir değişkene atanıp -1 ile karşılaştırıldı. 3 için 6, -3 için "Sonuç hatalı" çıktısı alındı.

> **Ders sonrası not:** Yukarıdaki `main`, iki çağrıyı bir arada gösterecek biçimde yazıldı. Kod `gcc -Wall` ile uyarısız derlendi ve aynı çıktıyı verdi.

3.3 Stack ile Adım Adım İzleme: $f(3)$

> **Hocanın vurgusu:** Özyinelemeli kodları adım adım takip etmenin ve analiz etmenin yolu stack kullanmaktır.

Stack'e ekleme de silme de üstten yapılır. $f(3)$ çağrıldığında:

- 3 sıfırdan küçük değil, sıfıra da eşit değil. `3 * f(2)` hesaplanmak üzere $f(2)$ çağrılır.
- $f(3)$ henüz bitmedi. Kaldığı yerden devam etmesi için stack'te saklanır.
- Aynısı $f(2)$ için (`2 * f(1)`) ve $f(1)$ için (`1 * f(0)`) olur.
- $f(0)$ sıfırdan küçük değildir ama sıfıra eşittir, bu yüzden 1 döndürür. Bu alt program tamamlanır ve stack'ten çıkar.

Geri dönüşte en son çağrılan ilk tamamlanır:

```
Doluş (üstteki en son eklendi)      Boşalış
+-----------+
| f(0)      |                       f(0) = 1
| 1 * f(0)  |                       1 * 1 = 1
| 2 * f(1)  |                       2 * 1 = 2
| 3 * f(2)  |                       3 * 2 = 6
+-----------+
```

Soru: Return yazınca döngüdeki `break` gibi mi oluyor? **Cevap:** Hayır. `return` başka, `break` başka bir şeydir.

Soru: `return 1`, `return 0`, `-1` kafamı karıştırdı. 1 olunca veya -1 olunca ne oluyor? **Cevap:** $0!$ değeri 1'dir. Kullanıcı $f(0)$'ı çağırsaydı başka işleme gerek kalmadan doğrudan 1 döndürülecekti. Bu aynı zamanda base case'tir, çünkü $f(5)$ hesaplanırken $5 \cdot f(4)$ diye devam edilir ve sonunda kaçınılmaz olarak $f(0)$'a ulaşılır. O yüzden $f(0)$'ın karşılığı yazılır. `n < 0` kontrolü ise -5 gibi faktöriyeli olmayan bir değer gelirse fonksiyonun devam etmemesi içindir. -1 döndürülür, çağıran taraf da sonucu kontrol edip hatalı veri mesajı yazar. Base case her problemde farklıdır, problemin matematiğine göre değişir.

Soru: Stack'in limiti heap'e göre düşük demiştik. Çok derin özyinelemeli çağrılar başarısız olur mu? **Cevap:** Bu konu önceki derste konuşuldu, o dersin kaydında bulunabilir.

> **Ek bilgi (derste anlatılmadı):** Stack sınırı aşıldığında büyük olasılıkla program "stack overflow" denen hatayla sonlanır, Linux'ta bu genellikle "segmentation fault" olarak görülür. Base case'i unutulmuş bir özyineleme de büyük olasılıkla bu şekilde biter. Çünkü her yeni çağrı stack'te yer kaplar ve bu yer sonludur.

3.4 Slayttaki Stack Örneği: $f(4)$

Hoca aynı mantığı ders notundaki bir slayttan da gösterdi. Önemli bir ayrıntı şudur: `main` de stack'tedir. Program çalışınca önce `main` stack'e alınır. `main`, `fakt(4)`'ü çağırınca kaldığı yer saklanır. Normal bir fonksiyonda olduğu gibi işlem bitince `return` ile 24 değeri çağrıldığı yere döner, `main` devam eder ve biter.

Özyinelemeli sürümde her çağrı stack'e kendi satırını ekler: `4 * f(3)`, `3 * f(2)`, `2 * f(1)`. Slaytta base case 1 alınmıştı. Sonra en son çağrılandan başlanarak geriye doğru toparlanır (ara adım eklendi): $f(1) = 1$, $2 \cdot 1 = 2$, $3 \cdot 2 = 6$, $4 \cdot 6 = 24$.

Hocanın benzetmesi iç içe küme parantezleridir: İlk açılan parantez en son kapanır, en son açılan ilk kapanır. Buna son giren ilk çıkar (last in first out) denir. Stack de bu mantıkla çalışır.

3.5 Faktöriyelin Analizi

İteratif algoritmada n tane çarpma olduğunu görmüştük. Özyinelemeli çağrıda değer birer birer azalır:

$$f(n) = n \cdot f(n-1) = n \cdot (n-1) \cdot f(n-2) = \cdots = n \cdot (n-1) \cdots 1$$

Yani fonksiyon yine n kez çağrılır ve n çarpma yapılır. İkisi de n çarpma yapar, ama özyinelemeli olanın bir dezavantajı daha vardır: fazladan stack kullanır. Fonksiyon çağrısı ek yük demektir. Hoca şu an için özyinelemenin pek bir işe yaramadığını söyledi.

Asıl fayda farklı algoritmalarda, özellikle sıralamada görülür. Selection sort, insertion sort, bubble sort gibi iteratif sıralamalar n^2'dir. Merge sort ve quick sort ise $n \log n$'dir. Hoca bunun yalnız özyinelemeden değil, böl ve yönet (divide and conquer) tarzı algoritmalardan geldiğini de ekledi. Bu, süre açısından çok ciddi bir iyileştirmedir. n'lik sıralamalar da vardır, ama counting sort gibi algoritmalar genel amaçlı bir sıralama olarak kullanılamaz.

4. Özyinelemeli Üs Alma

4.1 Bağıntı

x^n hesabı yapacağız. $x \neq 0$ için $x^0 = 1$, ayrıca $x^1 = x$'tir. Tabanlar aynıyken üsler toplandığı için $x^n = x \cdot x^{n-1}$ yazılabilir. Bu, fonksiyonu üssü birer birer azaltarak çağırmamızı sağlar. Negatif üs için $x^{-n} = \left(\frac{1}{x}\right)^n$ kullanılır. Hoca bağıntının dört duruma çıktığına dikkat çekti:

$$p(x,n) = \begin{cases} 1 & n = 0,\ x \neq 0 \\ x & n = 1 \\ p\left(\frac{1}{x}, -n\right) & n < 0 \\ x \cdot p(x, n-1) & \text{diğer durumlarda} \end{cases}$$

> **Hocanın vurgusu:** Bu tarz programların en önemli kısmı recurrence relation'ı yazabilmektir.

4.2 Örnek: 2^3

Verilen: $p(2,3)$. Durum dördüncü satırdır.

- $p(2,3) = 2 \cdot p(2,2)$
- $p(2,2) = 2 \cdot p(2,1)$
- $p(2,1)$ base case'e düşer ve 2 döndürür.

Geri dönüş: $2 \cdot 2 = 4$, sonra $2 \cdot 4 = 8$. Sonuç 8'dir.

4.3 Örnek: 2^{-3}

Verilen: $p(2,-3)$. n sıfırdan küçüktür, bu yüzden $p\left(\frac{1}{2}, 3\right)$ çağrılır. Problem artık pozitif üslü yeni bir probleme dönüşmüştür:

- $p\left(\frac{1}{2}, 3\right) = \frac{1}{2} \cdot p\left(\frac{1}{2}, 2\right)$
- $p\left(\frac{1}{2}, 2\right) = \frac{1}{2} \cdot p\left(\frac{1}{2}, 1\right)$
- $p\left(\frac{1}{2}, 1\right)$ base case'e düşer ve $\frac{1}{2}$ döndürür.

Geri dönüş: $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$, sonra $\frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$. Sonuç $\frac{1}{8}$'dir.

Soru: Slaytta anlatırken "eksi bir bölü iki" demiştiniz. **Cevap:** Orada yanlış yazılmış. Doğrusu $\frac{1}{2}$ ve $\frac{1}{2^3}$'tür, o eksi bir yoktur.

4.4 Kod

Hoca önce iteratif sürümü yazdı: `r` 1'den başlar, x kendisiyle n defa çarpılır. Hazır üs fonksiyonu olduğunu söyleyenler için de burada algoritmanın tasarlandığını hatırlattı. Hocaya göre iteratif çözümü düşünmek nispeten zordur, matematiksel olarak kurgulanınca özyinelemeli fonksiyon daha kolay bir algoritmadır. Aynı şeyin Fibonacci'de de konuşulacağını ekledi.

Özyinelemeli sürümü yazarken karşılaşılan noktalar şunlardır:

- `n == 0` durumu, 0^0 olmasın diye $x \neq 0$ şartıyla birlikte en başa yazıldı.
- `n == 1` asıl temel durumdur. Programın sonsuza kadar çağrılmaması buna bağlıdır.
- Negatif üste `1 / x` yazılırsa tam sayı bölmesi yapılır. Bu C'ye ait bir ayrıntıdır. Bu yüzden `1.0 / x` yazıldı.
- Sonuç artık tam sayı olamaz. Fonksiyonun dönüş tipi `double`'a çevrildi ve `printf`'te `%d` yerine `%lf` kullanıldı.

- Derlemede küçük harf-büyük harf hatası ve fonksiyonun `main`'den sonra tanımlanmasından doğan hata görüldü. `P` küçük harfle düzeltildi, fonksiyon yukarı alındı.

  Hocanın vurgusu: En temel durumları, istisnai durumları fonksiyonun başına yazmak en doğrusudur. Alt taraflara yazılırsa diğer çağrılar yüzünden işler karışabilir, yoksa ayrıca kontroller yapmak gerekir.

```c
#include <stdio.h>

double p(double x, int n)
{
    if (n == 0 && x != 0)
        return 1;
    if (n == 1)
        return x;
    if (n < 0)
        return p(1.0 / x, -n);
    return x * p(x, n - 1);
}

double pi(double x, int n)      /* iteratif sürüm */
{
    double r = 1;
    int i;

    for (i = 1; i <= n; i++)
        r = r * x;
    return r;
}

int main(void)
{
    printf("%lf\n", p(2, 3));
    printf("%lf\n", p(2, 0));
    printf("%lf\n", p(2, -3));
    printf("%lf\n", pi(2, 3));
    return 0;
}
```

Derste $p(2,3)$, $p(2,0)$ ve $p(2,-3)$ için 8, 1.0 ve 0.125 sonuçları alındı.

> **Ders sonrası not:** Yukarıdaki kod `gcc -Wall` ile uyarısız derlendi ve aynı değerleri verdi, iteratif sürüm de 2^3 için 8 verdi. Hoca iteratif döngüyü konuşurken "birden, küçüktür n'e kadar" dedi. `r` 1'den başlayıp `i` 1'den `i < n` ile dönseydi $n-1$ çarpma yapılırdı. Hoca hemen sonra "n defa çarpıyoruz" dediği için yukarıda `i <= n` yazıldı. Ekrandaki koşul büyük olasılıkla ya bu biçimdeydi ya da `i` sıfırdan başlıyordu.

4.5 Negatif Üs Dalında `return` Sorusu

Soru: 17. satırı anlamadım. Fonksiyon değer üretiyor, orada `p` nasıl boşta duruyor? **Cevap:** Hocanın ekrandaki kodunda negatif üs dalı `p(1.0 / x, -n);` biçimindeydi, başında `return` yoktu. Hoca, fonksiyonun orada değer üretmediğini, içinden yeni bir fonksiyon çağrısı yaptığını, 2^{-3}'ün çarpmayla doğrudan hesaplanamadığı için problemin $\frac{1}{2^3}$ haline dönüştürüldüğünü söyledi. `return` yazılırsa bir recursive çağrının fazladan yapılacağını, sonucun aynı olacağını ama buna gerek olmadığını ekledi.

> **Ders sonrası not:** Hoca `return` yazılırsa fazladan bir çağrı olacağını söyledi; doğrusu çağrı sayısının değişmemesi ve `return`'ün gerekli olmasıdır. Ders sonrasındaki denemede `return`'süz sürüm `gcc -Wall` ile "control reaches end of non-void function" uyarısı verdi. Yine de 0.125 çıktısı alındı. `return` olmayınca iç çağrının sonucu kullanılmaz ve fonksiyon değer döndürmeden biter. Bunun doğru çıkması büyük olasılıkla dönüş değerinin yazmaçta tesadüfen kalmasındandır. Başka bir derleyicide

veya ayarda büyük olasılıkla yanlış sonuç alınabilir. Doğru yazım yukarıdaki `return p(1.0 / x, -n);` satırıdır.

4.6 Analiz

İteratif kodda 1'den n'e kadar n çarpma vardır. Özyinelemeli kodda da 3'ten 2'yi, 2'den 1'i çağırdığımız gibi n'den $n-1$'e, $n-1$'den $n-2$'ye inilir, yani yine n çarpma vardır. İkisi de n çarpma yapar.

5. Daha İyi Bir Üs Alma Fikri

Hoca aradan önce şu soruyu sordu: Daha iyi bir algoritma olur mu? Bildiğimiz şu eşitlikler kullanılabilir:

$$x^n = \begin{cases} x^{n/2} \cdot x^{n/2} & n \text{ çift} \\ x^{\lfloor n/2 \rfloor} \cdot x^{\lfloor n/2 \rfloor} \cdot x & n \text{ tek} \end{cases}$$

Burada $n/2$ aşağıya yuvarlanır. Örneğin x^7 hesaplanırken $x^7 = x^3 \cdot x^3 \cdot x$ yazılır. Önceki yöntemde x^7 için $x \cdot x^6$ bulunuyordu, yani 6'ya iniliyordu. Burada bir anda x^3'e geçildi. x^3 bir kez hesaplanırsa tekrar hesaplanması gerekmez: kendisiyle çarpılır, bir de x ile çarpılır. Böylece problem 6'dan 3'e inmiş olur.

6. Üs Almanın Diğer Yazımları

Aradan dönüşte hoca ekranda aynı işin farklı yazımlarını gösterdi.

- **Birinci yazım:** En temel yöntem. $n = 1$ ise x'i döndür, değilse n'i birer azaltarak çağır.
- **İkinci yazım (yalnız negatif üsler için):** $n = 0$ ise 1 döndür, değilse `1.0 / x * p(x, n + 1)` döndür. Üs -3 ise -2, -1, 0 diye yukarı doğru gider. Pozitif üs verilirse sıfıra hiç ulaşmaz. Hocanın örneğiyle x^5 verildiğinde üs 6, 7 diye büyür. Hoca bu yüzden bu yazımın pozitif üsler için çalışacak bir şey olmadığını söyledi.
- **Üçüncü yazım (bütün durumlar için):** $n = 0$ ise 1 döndürür. $n < 0$ ise `1.0` bölü olarak başa konur, yani bölme bütün sonuca uygulanır. Üs -3'ten geriye gitmek yerine ileriye doğru gidilir. Aynı sayıda çarpma yapılır ve toplam sonuç `1.0`'a bölünür. Değilse klasik $x \cdot p(x, n-1)$ çalışır. Hoca olması gerekenin bu genel yazım olduğunu söyledi.

  Ders sonrası not: İkinci yazım derlenip denendiğinde $p(2, -3)$ için 0.125 verdi. Pozitif üs ile yapılan denemede sıfıra ulaşmadı, sayaçla kesilene kadar kendini çağırdı. Üçüncü yazımın negatif dalı, anlatımdan kurulan `return 1.0 / (x * p(x, n + 1));` biçiminde denendiğinde $p(2, -3)$ için 0.125 yerine 0.5 verdi, çünkü bölme her adımda tekrar uygulanıyor. Ekrandaki kod büyük olasılıkla bölmeyi yalnız en dışta uyguluyordu. Bölmeyi tek kez uygulayan `return 1.0 / p(x, -n);` yazımı denemede 0.125 verdi.

7. Ders Notundan Üç Fonksiyon

Hoca, Fatih Hoca'nın ders notundaki sözde kodla (pseudocode) yazılmış üç fonksiyonu sordu: Bunlar ne iş yapar?

7.1 `f(a, b)`: Toplama

```
T = f(a, b)
    if b == 0
        T = a
    else
        T = f(a + 1, b - 1)
    end
```

Soru: $f(12, 4)$'ün sonucu nedir? **Cevap:** Bir öğrenci 16 dedi ve fonksiyonun sayıları topladığını söyledi. Hoca bunu adım adım gösterdi:

- $f(12, 4)$: b sıfır değil, $f(13, 3)$ çağrılır.
- $f(13, 3)$: b sıfır değil, $f(14, 2)$ çağrılır.
- $f(14, 2)$: b sıfır değil, $f(15, 1)$ çağrılır.
- $f(15, 1)$: b sıfır değil, $f(16, 0)$ çağrılır.
- $f(16, 0)$: $b = 0$, `T = a` yani 16 olur.

Bundan sonra yapılacak başka iş yoktur. Her çağrı `end`'e gelir ve 16 değerini bir üst çağrıya döndürür. Sonuç 16'dır. Fonksiyon $b > 0$ için $a + b$'yi hesaplar.

7.2 `f(dizi, b)`: Dizi Toplamı

İkinci fonksiyon bir dizi alır. Eleman ilk elemansa doğrudan onu döndürür. Değilse bulunduğu elemanı, bir önceki eleman için yapılan çağrının sonucuyla toplar. Yani dizideki bütün elemanları özyinelemeyle toplar: `dizi(b) + dizi(b-1) + dizi(b-2) + ....`

7.3 `f(dizi, b, x)`: Sıralı Arama

Üçüncü fonksiyon bir dizi, eleman sayısı b ve aranan x değerini alır. b birden küçükse böyle bir eleman yoktur ve hata göstermek için -1 döndürülür (hoca C'de sıfırdan yazılması gerektiğini ekledi). `dizi(b)` x'e eşitse b döndürülür. Değilse $b - 1$ için tekrar çağrılır. Yani dizinin 5., 4., 3., 2. elemanına bakılarak bütün dizi dolaşılır.

7.4 Üçünün Karşılaştırması

Hoca üç fonksiyonu klasik çözümleriyle karşılaştırdı.

Soru: İlk fonksiyon a ve b'ye göre kaç işlem yapar? **Cevap:** 3 değil. b'nin sıfıra gelmesi beklenir, yani b defa (n defa) işlem yapılır. Oysa $a + b$ tek bir toplamayla sabit zamanda hesaplanabilir. Doğrusal bir işlem sabit zamanlı bir işin yerine geçtiği için bu kötü bir algoritmadır.

Soru: Bir dizinin bütün elemanlarını toplamak kaç işlem gerektirir? **Cevap:** $2n$ olmaz. Dizi başından sonuna bir kez dolaşılır, yani n işlem gerekir. Özyinelemeli sürüm de b. elemandan başlayıp ilk elemana kadar toplayarak gelir. Eleman 10 ise 10 defa, 100 ise 100 defa çağrılır. Süre yine n'dir.

Soru: Sıralı aramada (sequential) en kötü durum nedir? **Cevap:** n defa. n^2 olmaz, çünkü o durumda dizi gereksiz yere tekrar dolaşılmış olurdu. Başından başlanır, son elemana kadar kontrol edilir. Özyinelemeli sürüm de b kez çalışır. İteratif sürümde `while` döngüsü ya bulana kadar ya da dizinin sonuna kadar gider.

Özetle bu üç örnekte özyineleme zamanı iyileştirmez. Bellek açısından ise bunlar daha kötü algoritmalardır, çünkü çağrıların bellek maliyeti vardır.

8. Recursion Tree (Özyineleme Ağacı)

Hoca bu örnekleri hızlıca konuşmasının amacının bu kavrama gelmek olduğunu söyledi. Recursion tree, özyinelemeli bir algoritmanın derinliğini ve toplam işlem sayısını bulmak için kullanılır. Özellikle böl ve yönet algoritmalarında zaman karmaşıklığını ölçmede işe yarar.

Faktöriyelin uç durumlar dışındaki genel hali $n \cdot f(n-1)$'dir. Her düğüm bir çarpma yapar ve bir alt düğüm çağırır. Çarpmanın maliyetini unutamayız.

```
f(n)      -> 1 çarpma
 |
f(n-1)    -> 1 çarpma
 |
f(n-2)    -> 1 çarpma
 |
...
```

```
 |
f(n-n) = f(0)
```

Ağaç n'den $n-n$'e kadar derinleşir, yani n düzey vardır. Her düzeyde 1 birim işlem olduğu için toplam $n \cdot 1 = n$ işlem yapılır. Karmaşıklığa n denir. Bu, iteratif algoritmanın değeriyle aynıdır.

Fibonacci'de $f(n-1) + f(n-2)$ bağıntısı vardır. Hoca orada recursion tree'nin pek işe yaramadığını, başka yöntemlerin anlatıldığını ve bunun gelecek dönem ayrık matematiğin konusu olduğunu söyledi.

Buraya kadarki bütün algoritmalarda ağaçtaki toplam işlem n çıktı ve fonksiyon çağrılarından dolayı zaman kaybı oldu. Ama özyinelemenin işe yarayan bir tarafı da vardır: üs almayı daha kısa sürede yapmak.

9. Hızlı Üs Alma: power_v2

9.1 Fikir

Ders notunda yazılan algoritmaya göre x^p hesaplanırken $p = 0$ ise sonuç 1'dir. p tekse sonuç, $x^{\lfloor p/2 \rfloor}$'nin karesi çarpı x'tir. p çiftse sonuç, $x^{p/2}$'nin karesidir. Önceki bölümdeki $x^{n/2} \cdot x^{n/2} \cdot x$ ifadesinde $x^{n/2}$ bir kez hesaplandıysa karesi alınır, tekse bir de x ile çarpılır.

Çift durumda x^n için $x^{n/2}$, onun için $x^{n/4}$ gerekir. İç içe gidilerek $x^{n/n} = x^1$'e ulaşılır. Bu noktadan, stack'te olduğu gibi geriye doğru toplanarak x^n daha kısa yoldan hesaplanır.

9.2 Kod ve Geçici Değişken

Hoca burada bir ara değişken (`temp`) kullandı. Çünkü $x^{n/2}$ bulunduktan sonra bir daha hesaplanması gerekmez. Hocanın ifadesiyle burada örtüşen problemler vardır. `power(x, y/2) * power(x, y/2)` yazmak gereksiz yere aynı alt problemi iki kez çözer. Bir kez elde edilen sonuç saklanır ve karesi alınır.

```
double power_v2(double x, int y)
{
    double temp;

    if (y == 0)
        return 1;
    temp = power_v2(x, y / 2);
    if (y % 2 == 0)
        return temp * temp;
    return temp * temp * x;
}
```

`y / 2` çağrısında sayının tek veya çift olması önemli değildir. Hocanın örnekleriyle x^5 için de x^4 için de x^2'ye, x^{17} için de x^{16} için de x^8'e ihtiyaç vardır. Tek olan durumda fazladan bir x eklenir. Hoca recurrence relation'ı da yazdı:

$$p(x,n) = \begin{cases} 1 & n = 0 \\ p(x, n/2)^2 & n \bmod 2 = 0 \\ p(x, \lfloor n/2 \rfloor)^2 \cdot x & n \bmod 2 = 1 \end{cases}$$

Hoca bu sırada böl ve yönet algoritmaların bir dezavantajı olduğunu, aynı işi tekrar tekrar yaptırma durumuna girdiğini ve `temp` değişkeniyle bundan kurtulduklarını söyledi.

> **Ders sonrası not:** Hoca bu dezavantajı böl ve yönet algoritmaları için genel olarak söyledi. Bu, büyük olasılıkla yalnız aynı alt problemin tekrar çözüldüğü durumlar için geçerlidir. Merge sort'ta iki yarı farklı elemanlardan oluştuğu için aynı iş tekrarlanmaz.

Hoca `temp` olmadan yazılırsa ne olacağını da anlattı. Önce birinci çağrı çalışır ve dallanarak gider. O dönünce ikinci çağrı aynı şeyi baştan yapar. x^{10} için x^5 bir yerde hesaplandıktan sonra bir daha hesaplanır. Bu çok kötüdür ve hesap yine n defaya çıkar.

10. Çağrı Sayısını Ölçme Denemesi

Hoca, kaç işlem yapıldığını görmek için 2^{10} üzerinde bir deneme yaptı. Sonuç 1024 çıktı. Normalde global değişken kullanmadıklarını, burada yalnız deneysel olarak göstermek için kullandığını belirterek `p` ve `power_v2` için iki sayaç tanımladı. Bu sayaçlar fonksiyonların içinde artırıldı ve tek bir `printf` ile yazdırıldı.

Ekranda önce 0 ve 4 gibi beklenmeyen değerler görüldü.

Soru: p fonksiyonunda sayacı `if`'ten önce artırsak daha doğru olmaz mıydı? **Cevap:** Olabilir, çünkü fonksiyona girildiği anda sayılmış olur.

Sayaçlar 1'den başlatılınca ekranda 1 ve 9 görüldü. Hoca, 10 olmasını beklediğini, 1 değerinin ise ilginç olduğunu söyledi. 2^{10} için 2^5'e, onun için 2^2'ye bakılacağını, yani logaritma mertebesinde 3 civarı bir değer beklediğini belirtti. Sonra bir yerde hata yapıldığını, ama o anki amacın özyinelemeli işlemi tasarlamak olduğunu söyleyip denemeyi bıraktı.

> **Ders sonrası not:** Ders sonrasında aynı deneme `gcc` ile tekrarlandı. Sayaçlar ve fonksiyonlar aynı `printf`'in argümanı olarak yazıldığında iki sayaç da 0 çıktı. Fonksiyonlar önce çağrılıp sonuçlar değişkene atandığında, sayaçlar ayrı okunduğunda `p` için 10, `power_v2` için 5 çıktı. Bunun sebebi büyük olasılıkla C'de argümanların hesaplanma sırasının belirli olmamasıdır: sayaç, fonksiyon çalışmadan önce okunmuş olabilir. `power_v2` için beklenen değer 3 değil 5'tir, çünkü özyineleme y değerleri 10, 5, 2, 1, 0 üzerinden iner. Bu da yine logaritmik mertebededir.

11. Üssün İkili Açılımıyla Üs Alma

Hoca, dönem başında anlattığı başka bir yöntemi de hatırlattı. Bu yöntemde geriye doğru değil ileriye doğru gidilir. Önce x^2, sonra x^4 hesaplanır. x^{10} için:

- x^2 hesaplanır.
- x^4 hesaplanır.
- x^5'e geçmek için bir çarpma daha yapılır.
- x^{10} için doğrudan karesi alınır.

Bu sıra, 10'un ikili açılımından gelir: $10 = 1010_2$. Hocanın ifadesiyle her bit, ya yalnız kare almayı ya da hem kare alıp hem x ile çarpmayı söyler.

> **Ders sonrası not:** Hoca x^4'ten x^5'e geçerken "kendisiyle çarpmak" dedi; doğrusu x ile çarpmaktır, $x^4 \cdot x = x^5$. Yukarıdaki sıra python ile denendi ve x^{10} değerini verdi.

12. Karmaşıklık Adları ve power_v2'nin Analizi

12.1 Her Çağrının Maliyeti

Hoca koda bakarak her çağrıda ne yapıldığını saydı: bir atama (`temp` hesabı), tek veya çift kontrolü ve bir çarpma. Toplama çıkarma gibi başka bir işlem yoktur. Bunları 3 diye saysak bile birim zamanda sabit (constant) bir işlem kabul edilir.

12.2 Karmaşıklık Adları

Hoca bu noktada arada karıştırılan adları netleştirdi:

- **Sabit zaman (constant):** Her zaman 1 ile gösterilir. 2 de, 5 de 1 kabul edilir. n^0 de 1 demektir.
- **Doğrusal zaman (linear):** n.
- **Karesel zaman (quadratic):** n^2.
- **Logaritmik zaman:** $\log n$.

- **Doğrusal ve logaritmik birlikte:** $n \log n$.

12.3 Ağaçla Derinliği Bulma

```
p(n)      -> 1 işlem
 |
p(n/2)    -> 1 işlem
 |
p(n/4)    -> 1 işlem
 |
...
 |
p(n/k)    (n/k = 1 olunca durulur)
```

k değeri 1, 2, 4 diye 2'nin kuvvetleri şeklinde ilerler, yani $k = 2^i$'dir. Duracağımız noktada:

$$\frac{n}{2^i} = 1 \quad \Rightarrow \quad n = 2^i$$

İki tarafın 2 tabanında logaritması alınır. $\log_2 2 = 1$ olduğu için:

$$\log_2 n = i \cdot \log_2 2 = i$$

Her derinleşmede 1 birim işlem olduğundan toplam $1 \cdot \log_2 n$'dir. Karmaşıklık $\log n$ olur. n defada yapılan işlem $\log n$'e inmiştir.

> **Ders sonrası not:** Ağaçta durma noktası $n/k = 1$ alındı, kodda ise `y == 0` kontrolü vardır, yani $y = 1$'den sonra bir çağrı daha yapılır. Ders sonrası sayaçlı denemede $n = 1024$ için 12 çağrı görüldü. Mertebe değişmez.

13. Geçici Değişken Olmadan: power_v3

Hoca kötü halini de yazıp sonucunu bulmak istedi. Artık `temp` yoktur, alt problem doğrudan iki kez çağrılır.

```
double power_v3(double x, int y)
{
    if (y == 0)
        return 1;
    if (y % 2 == 0)
        return power_v3(x, y / 2) * power_v3(x, y / 2);
    return power_v3(x, y / 2) * power_v3(x, y / 2) * x;
}
```

Hocanın örneğiyle 2^8 dediğimizde 2^4 iki kez çağrılır. İlk 2^4 kendi içinde 2^2'yi, o da 2^1'i çağırır. Hesaplanan şey tekrar tekrar hesaplanmış olur. Alt satırın üst satırdan tek farkı fazladan x çarpanıdır.

> **Ders sonrası not:** Hoca çift durumu yazarken "tek sayıysa, yani y mod 2 eşittir sıfırsa" dedi; doğrusu çifttir. Aynı satırda duyulan "x bölü x" ifadesi büyük olasılıkla dil sürçmesidir, çağrı `power_v3(x, y / 2)` olmalıdır. Hoca benzer bir yazımı `power_v2` bağıntısında "x bölü 2 yanlış yazdık" diyerek düzeltmişti.

Hoca önce maliyeti şöyle açtı: bir işlem artı $p(x, n/2)$ artı $p(x, n/2)$. Burada maliyetler çarpılmaz, toplanır, çünkü ağaç ikiye ayrılır. Önce her dalın $\log n$ getireceğini, iki $\log n$'in toplamının yine $\log n$ olacağını, zamanın aynı kalıp yalnız belleğin kötüleşeceğini söyledi. Hemen ardından bunun yanlış olduğunu fark etti ("log n olmaz") ve ağacı baştan çizdi.

```
Düzey 0:              p(n)                      2^0 işlem
Düzey 1:       p(n/2)       p(n/2)              2^1 işlem
Düzey 2:    p(n/4) p(n/4) p(n/4) p(n/4)         2^2 işlem
Düzey 3:    p(n/8) ... (8 düğüm) ...            2^3 işlem
```

```
...
Düzey h:    n/n'e inen düğümler                2^h işlem
```

Her düğümde 1 işlem vardır, düzey başına düğüm sayısı 2^i'dir. Derinlik $n/2^h = 1$ koşulundan $h = \log_2 n$'dir. Toplam maliyet:

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i$$

Hoca, iteratif algoritmalarda gördükleri geometrik toplam formülünü hatırlattı:

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1}$$

Burada $a = 2$ ve üst sınır $\log_2 n$'dir:

$$T(n) = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 2 \cdot 2^{\log_2 n} - 1 = 2n - 1$$

$2^{\log_2 n} = n$ olduğu için önden gelen 2 ile $2n$ elde edilir, eksi 1 kalır, payda 1'dir. Sonuç olarak işlem sayısı n mertebesindedir. `temp` kullanıldığında sonuç $\log n$'di, şimdi n'e çıktı. Sebep, aynı işin tekrar tekrar yapılmasıdır.

> **Ders sonrası not:** İki sürüm `gcc -Wall` ile uyarısız derlendi, 2^{10} için ikisi de 1024 verdi. Sayaçlı denemede $n = 1024$ için `power_v2` 12, `power_v3` 4095 kez çağrıldı. Formül bu n için $2n - 1 = 2047$ verir. Fark büyük olasılıkla kodun `y == 0`'a kadar inmesinden gelir, mertebe yine n'dir. Stack derinliği iki sürümde de 12 çıktı. Yani derste önce söylenen "bellek kötüleşir" ifadesinin aksine, bu denemede asıl kayıp zamandır.

14. Merge Sort ve Quick Sort

14.1 İki Sıralı Diziyi Birleştirme

Hoca sıralama algoritmalarından iki örnek olarak merge sort ve quick sort'u ele aldı. Önce daha önce konuşulan birleştirme (merge) işlemini hatırlattı: elimizde iki sıralı dizi vardır, karşılaştırılarak sıradaki eleman bunlardan birinden alınır. Elemanlar n ve m ise $n + m$ işlem yapılır. İkisine de n dersek $2n$ olur, yani n mertebesinde bir karmaşıklıktır.

> **Ders sonrası not:** Hoca "hangisi büyükse ilk önce ondan alıyorduk" dedi; derste yapılan artan sıralama örneğinde $(6, 8)$ doğrusu küçük olanı önce almaktır.

14.2 Merge Sort: 8, 6, 3, 5 Örneği

Merge sort, diziyi bölebildiği kadar böler.

```
Bölme:       8 6 3 5
            /       \
         8 6         3 5
        /   \       /   \
       8     6     3     5

Birleştirme: 8 + 6 -> 6 8
             3 + 5 -> 3 5
             6 8 + 3 5 -> 3 5 6 8   (ara adım eklendi)
```

Önce bölme (divide, partition) yapılır: 8 ve 6 bir alt dizi, 3 ve 5 başka bir alt dizidir. Sonra hepsi ayrı ayrı bölünür ve birer elemanlı dört dizi elde edilir. Ardından birleştirme algoritmasıyla

önce 8 ve 6 birleştirilip $6, 8$ yapılır, sonra 3 ve 5 birleştirilip $3, 5$ yapılır, en son bu iki küçük dizi birleştirilir.

Dizi yarı yarıya bölündüğü için 8 elemanlı dizi $4 + 4$'e, 16 elemanlı dizi $8 + 8$'e bölünür ve bir elemana ulaşılana kadar devam edilir. **Soru:** Kaç kez bölünür? **Cevap:** $\log n$ kez. Her bölme düzeyinde birleştirmede n işlem yapıldığı için karmaşıklık $n \log n$ olur. Bu, n^2'ye göre çok iyidir. Ama merge sort ek bellek kullanır.

Hoca, quick sort'un bu işin ek bellek kullanmayan hali olduğunu, onun da $n \log n$ olduğunu, algoritmasını bir ara anlatacağını söyledi. Kodlamaları ise daha çok Veri Yapıları ve Algoritmalar dersinin konusudur.

> **Ders sonrası not:** Hoca quick sort'u $n \log n$ olarak verdi. Bu, büyük olasılıkla ortalama durum içindir. Pivot hep en küçük veya en büyük eleman seçilirse quick sort büyük olasılıkla n^2'ye çıkar. Aşağıdaki bağıntı da quick sort için büyük olasılıkla yalnız diziyi iki eşit parçaya bölen durumda geçerlidir. "Ek bellek kullanmaz" ifadesi de büyük olasılıkla ek dizi kullanmamak anlamındadır, özyineleme için stack yine kullanılır.

14.3 Recursion Tree ile $n \log n$

Hoca iki algoritma için de aynı recurrence relation'ı verdi:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + n$$

İki tane $T(n/2)$, diziyi her adımda iki eşit parçaya bölmek demektir. Sondaki n ise her parçada yapılan temel işlemdir. Bu sabit değildir: algoritmanın doğası gereği orada `for` döngüsü gibi n işlemlik bir kısım vardır. Bu ayrıntılar Algoritma Analizi dersinde incelenir.

```
Düzey 0:   T(n)                        2^0 tane n      -> n
Düzey 1:   T(n/2)  T(n/2)              2^1 tane n/2    -> n
Düzey 2:   T(n/4) x 4                  2^2 tane n/4    -> n
Düzey 3:   T(n/8) x 8                  2^3 tane n/8    -> n
...
Düzey h:                               2^h tane n/2^h  -> n
```

Her düzeyin toplam maliyeti n'dir. Derinlik $\frac{n}{2^h} = 1$ eşitliğinden $h = \log_2 n$'dir. Bu, biraz önceki "böle böle gitmenin" sayısıdır. $\log n$ tane n toplanırsa:

$$T(n) = n \cdot \log_2 n$$

Yani $n \log n$ karmaşıklığına ulaşılır. Hoca Fibonacci analizine henüz girmediklerini, sonraki derste ayrıca ele alacaklarını belirtti.

> **Hocanın vurgusu:** Derste yapılan örneklerin kodlarını yazın ve adım adım inceleyin.

15. Laboratuvardaki Özyineleme Soruları

Geçen hafta asistanlar diğer gruba bir uygulama yapmıştı. Hoca oradaki soruların çözümlerini slayttan hızlıca gösterdi ve kodların paylaşılacağını söyledi. Aşağıdaki program, hocanın sözlü anlatımından kurulmuştur.

```c
#include <stdio.h>

int uzunluk(char *s)
{
    if (*s == '\0')
        return 0;
    return 1 + uzunluk(s + 1);
}
```

```c
int basamak(int sayi)
{
    if (sayi > 0)
        return 1 + basamak(sayi / 10);
    return 0;
}

int toplam(int n)
{
    if (n == 0)
        return 0;
    return n + toplam(n - 1);
}

int obeb(int a, int b)          /* Öklid: kalan ile */
{
    if (b == 0)
        return a;
    return obeb(b, a % b);
}

int obeb_cikarma(int a, int b)  /* laboratuvar çözümü: çıkarma ile */
{
    if (a != b) {
        if (a > b)
            return obeb_cikarma(a - b, b);
        return obeb_cikarma(a, b - a);
    }
    return a;
}

void tersyaz(char *s)
{
    if (*s != '\0') {
        tersyaz(s + 1);
        printf("%c", *s);
    }
}

int main(void)
{
    printf("%d\n", uzunluk("Ahmet"));
    printf("%d\n", basamak(123));
    printf("%d\n", toplam(10));
    printf("%d %d\n", obeb(16, 6), obeb_cikarma(16, 6));
    tersyaz("yaz");
    printf("\n");
    return 0;
}
```

Ders sonrası not: Program `gcc -Wall` ile uyarısız derlendi. Çıktılar sırasıyla 5, 3, 55, 2 2 ve zay oldu.

15.1 String Uzunluğu

String bir karakter dizisidir ve sonunda null karakter (`'\0'`) vardır. “Ahmet” yazıldığında `t`’den sonra bu karakter gelir. Pointer dizinin ilk elemanını gösterir. Fonksiyon her çağrıda pointer aritmetiğiyle bir sonraki elemana geçer, null karaktere ulaşınca durur ve her çağrıda 1 ekler. “Ahmet” için çağrılar “Ahmet”, “hmet”, “met”, “et”, “t” şeklinde gider. Sonuç 5’tir.

15.2 Basamak Sayısı

Klasik yöntemde sayı `while` ile sürekli 10'a bölünüyordu. Burada döngüyü özyineleme sağlar. Sayı sıfırdan büyükse 1 eklenir ve sayı 10'a bölünerek fonksiyon tekrar çağrılır. 123 için 123, 12, 1 değerleri geçilir, 1 bölü 10'dan 0 elde edilince durulur. Sonuç 3 basamaktır.

15.3 1'den n'e Toplam

Fonksiyon n'den başlayıp geriye doğru toplar. 1'den 10'a kadar dendiğinde önce 10, sonra 9, sonra 8 eklenir. Hoca bunun başka türlü de yazılabileceğini söyledi: Bir `i` parametresi 1'den başlatılıp n'e ulaşana kadar ileri doğru gidilebilir.

15.4 OBEB

Hoca önce Öklid algoritmasını $\gcd(16, 6)$ örneğiyle tekrar etti. Küçük sayı ilk parametreye yazılır, ikinci parametreye iki sayının bölümünden kalan yazılır:

$$\gcd(16, 6) = \gcd(6, 4) = \gcd(4, 2) = \gcd(2, 0) = 2$$

16'nın 6'ya bölümünden kalan 4, 6'nın 4'e bölümünden kalan 2, 4'ün 2'ye bölümünden kalan 0'dır. 0'a ulaşınca OBEB 2 olur.

Laboratuvar çözümünde bölme yerine çıkarma kullanılmıştır: $a \neq b$ iken a büyükse $(a - b, b)$, değilse $(a, b - a)$ çağrılır. Hoca sonucun değişmediğini, asıl Öklid algoritmasının kendisinin yazdığı olduğunu söyledi.

15.5 String'i Tersten Yazdırma

Özyinelemeli işlemler geriden başa doğru toparlandığı için bu özellik kullanılır. Fonksiyon önce bir sonraki eleman için kendini çağırır, ekrana yazma işi çağrıdan sonradır.

"yaz" için çağrılar "yaz", "az", "z" ve null karakter şeklindedir. Null karakterde şart yanlış olur ve bu çağrı hiçbir şey yazmadan döner. Sonra geriye doğru önce `z`, sonra `a`, sonra `y` yazılır. Çıktı "zay"dır.

Hoca bu örneklerin stringler konusunda özyinelemeyle birlikte tekrar işleneceğini söyledi.

16. Sonraki Konular

Hoca kalan haftaların akışını anlattı. Sıradaki konu, iteratif algoritmalar analiz edildikten sonra özyinelemeli algoritmaların analizidir. Ardından stringler ve şifreleme, sonra bağlı listeler (linked list) ve en son uygulamalar gelecek.

17. Final Projesinde Beklenen Algoritma Analizi

Dersin sonunda hoca final projesini anlattı. Bu notta yalnız algoritma analiziyle ilgili akademik beklenti aktarılmıştır.

17.1 Amaç

Derste görülmemiş bir algoritma seçilir, C dilinde yazılır ve karmaşıklığı zaman ve bellek açısından incelenir. Hoca kendiniz bir algoritma oluşturun demediklerini, var olan algoritmaları analiz edin dediklerini vurguladı. Algoritma bir kitaptan veya internet kaynağından alınıp incelenebilir.

Algoritma bu derste ve sayısal analiz dersinde görülenlerden, ya da onlara yakın olanlardan seçilmez. Örneğin selection sort anlatıldığı için başka bir sıralama algoritması çalışılması istenmedi. Quick sort ve merge sort da kabul edilmez. Hoca heap sort'un belki olabileceğini de ekledi. Amaç, dersin kapsamına girmeyen ve araştırma gerektiren bir algoritmayı öğrenmektir.

17.2 Otomatik Ölçüm

Çalışma zamanı ve bellek gereksinimi gibi ölçütlerden en az biri incelenir. Bu analiz kodun içinde otomatik olarak yapılır: ne kadar zaman harcandığı, kaç adımda bitirildiği veya ne kadar bellek kullanıldığı program tarafından gösterilir.

Kod farklı girdi boyutları için çalıştırılır, örneğin 100, 1000, 10.000 eleman. Algoritmaya göre boyutu değişen şey bir dizi, matris veya string olabilir. Hoca derste yapılan pi hesabını hatırlattı: Orada da 10^1, 10^2, 10^3 diye artırıldığında sonucun nasıl değiştiğine bakılmıştı.

> **Ek bilgi (derste anlatılmadı):** C'de çalışma süresini kod içinde ölçmek için büyük olasılıkla en kolay yol `time.h` içindeki `clock()` fonksiyonudur. Ölçülecek kısmın önünde ve arkasında çağrılır, fark `CLOCKS_PER_SEC`'e bölünür. Adım sayısı için ise bir sayaç kullanılabilir, derste yapılan deneme bunun bir örneğidir. Sayaç okunurken 10. bölümdeki `printf` sorununa dikkat edilmelidir.

17.3 Karşılaştırma Nedir?

Soru: Bir algoritma seçtik, karşılaştırmayı nasıl yapacağız? **Cevap:** Karşılaştırma normalde şudur: Eleman boyutuna göre algoritmanın çalışma hızı veya bellek durumu nasıl değişiyor? Var olan başka bir algoritmayla da karşılaştırılabilir, ama o algoritma herkesçe çok bilinen bir algoritma olmalıdır. Hocanın örneği doğrusal arama (linear search) ile ikili aramadır (binary search): biri n defada yapıyor, öteki $\log n$ defada yapıyor. İki rastgele algoritmayı doğrudan karşılaştırmak istenmez. Farklı veri boyutları için elde edilen sonuçların karşılaştırılması yeterlidir.

Soru: Yani bu kodda mı olacak? **Cevap:** Evet, kodda otomatik olarak olacak.

17.4 Program, Rapor ve Video

Program modüler olmalı, fonksiyonlarla yazılmalıdır. Derste görüldüğü gibi global değişken ve `goto` kullanılmaz, `break` yalnız `switch-case` içinde kullanılabilir.

Raporda algoritmanın tarifi, çalışma prensibi, kullanıldığı alanlar, kısıtları ve karmaşıklığı yer alır. 5-7 dakikalık sesli videoda ise algoritma kısaca tanıtılır, kod anlatılır ve analiz sonuçları (eksikleri, fazlaları, ne kadar zamanda bittiği, karmaşıklığı) açıklanır.

Dersin Toparlaması

Bu derste doktor sorusu tamamlandı ve özyineleme iki aşamalı bir tasarım olarak ele alındı: bağıntıyı yazmak ve base case'i belirlemek. Faktöriyel, basit üs alma, iki sayıyı toplama, dizi toplamı ve sıralı arama örnekleri, özyinelemenin tek başına kazanç sağlamadığını gösterdi. Bunlarda işlem sayısı yine n'dir, üstelik stack maliyeti eklenir.

Kazanç, problemi yarıya bölmekten gelir. Hızlı üs almada `temp` ile $\log n$'e inildi, `temp` kaldırılınca ağaç dallandı ve $2n - 1$ işlemle tekrar n'e çıkıldı. Merge sort'ta $T(n) = 2T(n/2) + n$ bağıntısı ağaçla çözülerek $n \log n$ bulundu. Sonraki ek derste özyinelemeli algoritmaların analizine devam edilecek. Fibonacci de ayrıca ele alınacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.